

Social Trend Mining: End-to-End Implementation Logic

1) System Scope and Layering

This repo implements a medallion-style pipeline for social trend mining:

- Bronze: raw Instagram-like JSON/JSONL files stored locally.
- Silver (`sl_*`): normalized, relational canonical model in Supabase/Postgres.
- Gold (`gl_*`): model outputs + trend/content/user KPI marts for analytics UI.

Design goals:

- Platform-agnostic IDs and schema (`platform:native_id`) to support Instagram now, and YouTube/TikTok later.
 - Rerunnable pipelines (UPSERT semantics) and run-versioned Gold outputs (`run_id`).
 - A/B comparability between analysis modes via `analysis_type` (`heuristic` or `ai`).
-

2) Bronze (Raw) Data Structure

2.1 Storage Convention

Raw files are kept in local REL folders (not in DB):

- `rel/data/ig/raw/*.jsonl`
- `rel/data/ig/raw/*.json`

Primary bulk input is line-delimited JSON (`.jsonl`), one post per line.

2.2 Raw Post Concepts (Instagram-like)

A raw post contains:

- Identity: post ID, user identity, username, follower count, account meta-data.
- Content: timestamp, caption, hashtags, mentions.
- Media: media type and properties.
- Engagement snapshots over time: cumulative metrics at day horizons (`t_day`).
- Ingestion metadata: ingest batch/date/source/query seeds.

Important modeling rule used throughout the project:

- No explicit structured location field is required in raw records.
- Region is inferred later from caption/hashtag cues.

3) Silver Layer (**sl_***): Canonical Relational Model

Silver DDL is implemented by:

- `supabase/migrations/20260309000100_sl_core.sql`

3.1 Canonical ID Strategy

Silver uses canonical IDs to isolate platform source specifics:

- `canonical_user_id = "<platform>:<native_user_id>"`
- `canonical_post_id = "<platform>:<native_post_id>"`

This makes cross-platform expansion straightforward while preserving native IDs.

3.2 Silver Tables and Roles

- `sl_users`: canonical user master.
- `sl_user_snapshots`: daily snapshots for follower/following/post base-lines.
- `sl_posts`: canonical post master (timestamp/content text/ingest meta-data).
- `sl_post_mentions`: normalized many-to-many mentions.
- `sl_tags`: deduplicated tags with normalized form.
- `sl_post_tags`: post-tag bridge.
- `sl_post_metrics_ts`: time-series engagement snapshots with dual time modes.

3.3 Tag Normalization

Helper function:

- `public.normalize_tag(input_tag) => lowercase + trim.`

Purpose:

- Consistent dedupe and joins for hashtag-driven discovery/trending.

3.4 Metrics Time Modeling

`sl_post_metrics_ts` supports two representations:

- `metric_time_type='age_day' + t_day` (e.g., D0/D7/D30 since publish)
- `metric_time_type='asof_date' + asof_date` (calendar snapshots)

A generated key (`metric_time_key`) and check constraints enforce valid combinations.

3.5 Why Silver Stays “Clean”

Silver intentionally avoids derived NLP/ML labels. It keeps factual normalized ingestion data only.

Examples not stored in Silver:

- language classification
- fashion/non-fashion decision
- sentiment/category/topic

These belong in Gold for run-versioning and model governance.

4) Gold Layer (gl_*): Predictions + Analytical Marts

Gold DDL is implemented by:

- supabase/migrations/20260309000200_gl_predictions.sql
- supabase/migrations/20260309000300_gl_trends.sql
- supabase/migrations/20260310000100_add_analysis_type_to_gl.sql
- supabase/migrations/20260310000200_add_breakout_streak_metrics.sql
- supabase/migrations/20260310000300_add_gl_kpi_tables.sql

4.1 Run Versioning and Analysis Type

- gl_runs is the run control table.
- run_id versions all downstream predictions and KPIs.
- analysis_type labels the inference path:
 - heuristic
 - ai

This supports side-by-side metric comparisons for the same date/category windows.

4.2 Per-Post Gold Predictions

gl_post_predictions stores post-level derived outputs:

- fashion flag + confidence
- language + confidence
- translation
- sentiment + confidence
- category/subcategory + confidence
- topic summary
- evidence (jsonb)
- analysis_type (heuristic|ai)

4.3 Horizon Metrics for Post Performance

`gl_post_metrics_horizon` stores selected horizon snapshots (e.g., d7) and normalized engagement score per post.

Core score formula:

- $\text{engagement_score} = (\text{likes} + 2 \cdot \text{shares} + 1.5 \cdot \text{comments}) / \text{followers_at_post}$

Notes:

- Shares get highest weight (virality signal).
 - Comments weighted above likes (higher effort signal).
 - Followers normalization enables fairer creator-size comparison.
-

5) Heuristic vs AI Processing Paths

5.1 Heuristic Path (`analysis_type='heuristic'`)

Rule-based logic infers language/fashion/category/sentiment/topic from lexical cues and simple thresholds.

Typical strengths:

- Fast, deterministic, low cost.
- Good for initial bootstrapping and pipeline validation.

Typical tradeoffs:

- Weaker contextual understanding.
- Lower robustness for multilingual nuance, slang, and implicit fashion references.

5.2 AI Path (`analysis_type='ai'`)

LLM-driven classification pipeline performs staged extraction:

1. Language detection
2. Translation to English
3. Fashion detection
4. Sentiment
5. Category/subcategory (if fashion)
6. Topic summary

`evidence` JSON stores supporting snippets/signals for traceability.

5.2.1 Category Guardrails

To avoid category explosion, category detection is constrained to a fixed top-level taxonomy:

- `modestwear`
- `streetwear`
- `accessories`
- `sportswear`
- `luxury`
- `capsule`
- `sustainable`
- `beauty_fragrance`
- `general_fashion` (fallback)

`general_fashion` acts as safe default when confidence is low or signals are diffuse.

6) Trend Construction in Gold

6.1 Clusters

`gl_trend_clusters` and `gl_post_cluster_membership` group posts into trend units.

MVP strategy used in this implementation:

- category-driven grouping (`cluster_method='category_group'`)

6.2 Daily Trend Metrics

`gl_trend_daily_metrics` includes per cluster/day:

- volume: `post_count`, `unique_authors`
- performance: `engagement_score_sum`, `engagement_score_avg`, `shares_sum`
- dynamics: `velocity_7d`, `acceleration_7d`, `breakout_score`, `threshold_hit`
- continuity: `breakout_streak`, `longest_breakout_streak`

Breakout concept:

- A continuous intensity score (`breakout_score`) compares current signal to recent baseline (z-score style).
- `threshold_hit` is a boolean crossing condition.

Streak concept:

- `breakout_streak`: consecutive threshold-hit days up to that date.
 - `longest_breakout_streak`: max observed streak up to that date.
-

7) KPI Marts and Meaning

7.1 Trend KPI Mart: `gl_trend_kpis_daily`

Grain:

- `(run_id, date_utc, cluster_id)`, with `analysis_type` and `category` attributes.

Key KPI families:

- Scale: `post_count`, `unique_authors`
- Engagement: `engagement_score_sum`, `engagement_score_avg`, `shares_sum`
- Breakout: `breakout_score`, `threshold_hit`, `breakout_days_30d`, `streak_fields`, `time_to_breakout_days`
- Growth: `post_growth_7d`, `engagement_growth_7d`
- Structure: `author_concentration_top10`, `novelty_score_7d`, `stability_score_14d`
- Composite: `trend_health_index`

How to interpret:

- High `breakout_score` + rising `post_growth_7d` => emerging trend.
- High `breakout_days_30d` + high `stability_score_14d` => sustained durable trend.
- High `author_concentration_top10` => trend driven by few creators (fragility risk).
- High `novelty_score_7d` => fresh creator inflow (trend expansion signal).

7.2 Content KPI Mart: `gl_content_kpis_daily`

Grain:

- `(run_id, date_utc, category)`, with `analysis_type`.

Core metrics:

- Raw totals: `likes_sum`, `comments_sum`, `shares_sum`, `saves_sum`, `views_sum`
- Normalized rates: `like_rate_per_1k`, `comment_rate_per_1k`, `share_rate_per_1k`, `save_rate_per_1k`
- Quality/composite: `engagement_quality_score`, `viral_ratio`

How to interpret:

- Rising `share_rate_per_1k` often indicates stronger organic spread than likes alone.
- Higher `save_rate_per_1k` often maps to utility/intent-rich content.
- `viral_ratio` tracks proportion of exceptional posts; useful for creative risk calibration.

7.3 User Influence Marts

gl_user_influence_daily

Daily user-level influence baseline by run/day.

gl_user_influence_7d_overall

7-day trailing ranking across all categories.

gl_user_influence_7d_by_category

7-day trailing ranking within each category.

Main score:

- `influence_score_7d = engagement_sum_7d * ln(1 + followers_est)`

How to interpret:

- High score with moderate follower base indicates efficiency/quality.
 - Compare overall vs category rank to distinguish broad vs niche influence.
-

8) AI KPI Build Logic (What Happens After LLM Predictions)

For `analysis_type='ai'`, KPI marts are computed from AI-labeled posts using the same metric engines as heuristic runs.

Flow:

1. AI labels write to `gl_post_predictions (run_id, analysis_type='ai')`.
2. Horizon extractor writes `gl_post_metrics_horizon` for selected horizon (e.g., d7).
3. Cluster builder groups fashion posts (category-driven in MVP).
4. Daily aggregators compute trend/content/user KPI marts.
5. Rankings are materialized for user influence (overall and per category).

Implication:

- Heuristic and AI runs are structurally comparable because they share downstream KPI formulas.
 - Differences are mostly due to upstream labeling quality/distribution.
-

9) Dashboard Dimensions and Slice Semantics

Primary slicers used by the app:

- Day/date range (`date_utc/window_end_date`)

- Category
- Analysis type (`heuristic` vs `ai`)

This enables:

- Temporal trend diagnostics
 - Taxonomy-level performance splits
 - Methodology comparisons (rule-based vs model-based)
-

10) Practical Interpretation Playbook

Use this sequence when reading results:

1. Start with trend scale and breakout:
 - `post_count`, `breakout_score`, `threshold_hit`, `streak` fields.
 2. Validate quality of momentum:
 - `engagement_growth_7d`, `share_rate_per_1k`, `engagement_quality_score`.
 3. Check concentration risk:
 - `author_concentration_top10`.
 4. Check sustainability:
 - `stability_score_14d`, `longest_breakout_streak`.
 5. Assign activation owners:
 - top creators from 7-day overall/category ranks.
-

11) Data Governance and Reliability Notes

- Gold outputs are run-versioned; always query with explicit `run_id`.
 - Use `analysis_type` for controlled comparisons.
 - Keep Bronze immutable and local for reproducibility.
 - Keep Silver factual and non-derived; keep derivations in Gold.
 - For frontend/API usage, prefer Supabase anon key with proper grants and RLS policies.
-

12) Key SQL Assets in This Repo

- Silver core schema: `supabase/migrations/20260309000100_sl_core.sql`
 - Gold prediction schema: `supabase/migrations/20260309000200_gl_predictions.sql`
 - Trend/influence schema: `supabase/migrations/20260309000300_gl_trends.sql`
 - Analysis type support: `supabase/migrations/20260310000100_add_analysis_type_to_gl.sql`
 - Breakout streak fields: `supabase/migrations/20260310000200_add_breakout_streak_metrics.sql`
 - KPI marts: `supabase/migrations/20260310000300_add_gl_kpi_tables.sql`
-

13) Silver Build Logic (Actual Loader Behavior)

Implementation files:

- `rel/silver_gold/pipeline/lib/bronze_reader.py`
- `rel/silver_gold/pipeline/lib/sl_loader.py`

Bronze reader validation:

- Requires `post_id`, `user`, `timestamp_utc`.
- Requires `user.user_id`.
- Raises hard errors on malformed lines.

Parallel transform behavior:

- Loader can run multi-process transforms (`ProcessPoolExecutor`) via `--workers`.
- Each transformed post is normalized into canonical records for users, posts, tags, mentions, and `t_day` metrics.

Upsert behavior (idempotent reruns):

- `sl_users`: upserts profile fields; updates `last_seen_at`.
- `sl_user_snapshots`: upserts by (`canonical_user_id`, `snapshot_date`).
- `sl_posts`: upserts by canonical post ID.
- `sl_post_mentions`: conflict-do-nothing on duplicate mention pairs.
- `sl_tags`: upserts by (`tag_type`, `tag_norm`) using `normalize_tag`.
- `sl_post_tags`: upserts mapping with positional index.
- `sl_post_metrics_ts`: upserts per (`canonical_post_id`, `metric_time_type`, `metric_time_key`).

Write pattern:

- Bulk `executemany` in chunks (`batch_size`, default 500), then final commit.

14) Heuristic Build: Exact Rules and Fallbacks

Implementation file:

- `rel/silver_gold/pipeline/lib/agents.py`

The heuristic path is not a placeholder. It has deterministic rules for each stage when LLM output is unavailable.

14.1 Language detection heuristic

Signals:

- Arabic script regex (`[\\u0600-\\u06FF]`)
- Latin alphabet presence

Decision:

- Arabic + Latin -> **ar-en**
- Arabic only -> **ar**
- Latin only -> **mixed**
- neither -> **other**

14.2 Translation heuristic

Behavior:

- Returns original text as **translation_en**.
- Sets **model='heuristic'**.

14.3 Fashion detection heuristic

Keyword pool:

- **fashion, style, ootd, abaya, look, fit, wardrobe, sneaker, bag, dress, modest, streetwear, luxury**

Decision:

- If any term appears in text/translation/hashtags -> **is_fashion=true**, confidence 0.85.
- Otherwise -> **is_fashion=false**, confidence 0.40.

14.4 Sentiment heuristic

Positive terms:

- **love, stunning, great, amazing, best, obsessed, nice**

Negative terms:

- **bad, worst, hate, awful, poor, disappointed**

Decision:

- Positive count > negative count -> **Positive**
- Negative count > positive count -> **Negative**
- Tie -> **Neutral**

14.5 Category heuristic

If **fashion=true**, checks tokens in order:

- **abaya -> modestwear/abaya**
- **sneaker -> streetwear/sneakers**
- **bag -> accessories/bags**
- **watch -> accessories/watches**
- **athleisure -> sportswear/athleisure_sets**

Fallback:

- `general_fashion/general` (with lower confidence) when no rule matches.

14.6 Category canonicalization safeguards

Even for AI outputs, values are constrained in code:

- Top-level categories must exist in `ALLOWED_CATEGORY_MAP`.
- Subcategory must belong to its parent category's allowed set.
- Invalid outputs are coerced to safe defaults.

This is the anti-fragmentation control that prevents exploding category cardinality.

15) Prompt Contracts (Stage Prompts + Batch Prompt)

Prompt folder:

- `rel/silver_gold/prompts/`

Single-stage prompt files:

- `lang_detect.md`
- `translate_en.md`
- `fashion_detect.md`
- `sentiment.md`
- `category_detect.md`
- `topic_summary.md`

One-pass batch prompt:

- `batch_enrich.md`

15.1 Single-stage prompt behavior

The synchronous agent path (`gl_run_agents`) calls prompts stage-by-stage in this fixed order:

1. `lang_detect.md`
2. `translate_en.md`
3. `fashion_detect.md`
4. `sentiment.md`
5. `category_detect.md` (only when `is_fashion=true`)
6. `topic_summary.md`

Each prompt enforces strict JSON output with bounded labels/confidence fields.

15.2 Batch prompt behavior

`batch_enrich.md` combines all six tasks in one prompt and returns one JSON object per post with:

- language + confidence
- translation + confidence
- fashion boolean + confidence
- sentiment + confidence
- category/subcategory + confidences
- topic summary + confidence
- evidence object

Taxonomy and defaults are embedded in the batch prompt to keep output deterministic.

15.3 Why both paths exist

- `gl_run_agents`: easy debugging and stage-level observability.
- Batch API path: lower cost and higher throughput for large post volumes.

Both write to the same Gold prediction table and share downstream KPI engines.

16) Gold Layer Build Order (Canonical Execution Sequence)

Pipeline entrypoint:

- `rel/silver_gold/pipeline/cli.py`

End-to-end starts with Silver load, then follows one of two prediction paths, then one shared KPI path.

Prerequisite step (Bronze -> Silver):

1. `sl_load --platform instagram --jsonl <bronze_file> [--workers N --batch-size M --offset 0 --limit L]`

16.1 Path A: Synchronous stage-by-stage (`gl_run_agents`)

1. `gl_run_agents --run-id <run> --analysis-type heuristic|ai`
2. `gl_compute_horizon --run-id <run> --horizon d7`
3. `gl_build_trends --run-id <run> --method category_group`
4. `gl_aggregate_daily --run-id <run>`
5. `gl_rank_users --run-id <run>`
6. `gl_compute_trend_kpis --run-id <run>`
7. `gl_compute_content_kpis --run-id <run>`
8. `gl_rank_users_7d --run-id <run>`

16.2 Path B: OpenAI Batch path (analysis_type='ai')

1. `gl_batch_prepare --run-id <run> --offset <n> --limit <m>`
2. `gl_batch_submit --input-jsonl <file>`
3. Poll: `gl_batch_status --batch-id <id>`
4. `gl_batch_download --batch-id <id> --out-jsonl <file>`
5. `gl_batch_ingest --run-id <run> --output-jsonl <file>`
6. Then run steps 2..8 from Path A (horizon -> trends -> aggregates -> KPIs).

Important:

- If you skip `gl_compute_horizon`, all KPI layers that rely on engagement normalization become sparse/zero-heavy.
 - If you skip `gl_build_trends`, trend KPI tables cannot populate correctly because cluster IDs are missing.
-

17) Batch API Technical Shape

Implementation file:

- `rel/silver_gold/pipeline/lib/batch_jobs.py`

17.1 Request JSONL generated by `gl_batch_prepare`

Per row format:

- `custom_id` = canonical post ID
- `method` = POST
- `url` = `/v1/responses`
- body includes:
 - `model`
 - `input` (rendered `batch_enrich.md`)
 - `reasoning.effort` = `minimal`
 - `text.verbosity` = `low`
 - `max_output_tokens` = `900`

17.2 Batch ingestion mapping

`gl_batch_ingest` reads each output line, extracts JSON from model text, and maps fields into:

- `gl_post_predictions` columns
- `analysis_type='ai'`
- evidence JSONB

Rows are upserted in bulk to keep reruns safe.

18) KPI Formulas (Code-Exact)

Core implementation files:

- `rel/silver_gold/pipeline/lib/trend_agg.py`
- `rel/silver_gold/pipeline/lib/kpi_agg.py`

18.1 Horizon engagement score

Used in `gl_post_metrics_horizon`:

- `engagement_score = (likes + 2*shares + 1.5*comments) / followers_at_post`

18.2 Trend breakout and streak logic

Daily base:

- aggregate `shares_sum` per day/cluster/platform.

Dynamics:

- `velocity_7d = shares_sum_today - shares_sum_prev_day`
- `acceleration_7d = velocity_today - velocity_prev_day`
- `breakout_score = (shares_sum - mean(prev14)) / std(prev14)` (0 if no variance)

Threshold:

- `threshold_hit = (shares_sum >= share_threshold) OR (breakout_score >= breakout_z)`

Continuity:

- `breakout_streak` increments on consecutive `threshold_hit=true`.
- `longest_breakout_streak` is running max over history.

18.3 Trend KPI composites

`trend_health_index` in `gl_trend_kpis_daily`:

- `0.40 * max(breakout_score, 0)`
- `+ 0.25 * post_growth_7d`
- `+ 0.25 * engagement_growth_7d`
- `+ 0.10 * novelty_score_7d`

Supporting metrics:

- `breakout_days_30d`: rolling count of threshold-hit days over last 30 calendar rows.
- `time_to_breakout_days`: days since first breakout date for cluster.
- `author_concentration_top10`: share of engagement from top 10 creators that day.

- `novelty_score_7d`: share of creators not seen in previous 7-day lookback.
- `stability_score_14d` = $1 / (1 + \text{stddev_14d}(\text{shares_sum}))$.

18.4 Content KPI formulas

Daily/category aggregates:

- `like_rate_per_1k` = $\text{likes_sum} * 1000 / \text{followers_sum}$
- `comment_rate_per_1k` = $\text{comments_sum} * 1000 / \text{followers_sum}$
- `share_rate_per_1k` = $\text{shares_sum} * 1000 / \text{followers_sum}$
- `save_rate_per_1k` = $\text{saves_sum} * 1000 / \text{followers_sum}$
- `engagement_quality_score` = $(\text{likes} + 1.5 * \text{comments} + 2 * \text{shares} + 2 * \text{saves}) / \text{followers_sum}$
- `viral_ratio`: fraction of posts with engagement score $\geq$ global p95 for that run slice.

18.5 User influence formulas

Daily baseline (`gl_user_influence_daily`) and 7-day windows (`gl_user_influence_7d_*`) use:

- `influence_score` = $\text{engagement_sum} * \ln(1 + \text{followers_est})$

7-day tables are rolling windows:

- overall ranking by `window_end_date`
- category ranking by (`window_end_date`, `category`)

19) Gold Run Separation: Heuristic vs AI (Operationally)

Gold data is separated by two fields together:

- `run_id`: identifies one pipeline execution/version.
- `analysis_type`: identifies inference mode (`heuristic` or `ai`).

Where separation is stored:

- `gl_runs.analysis_type`
- `gl_post_predictions.analysis_type`
- KPI marts also carry `analysis_type`:
 - `gl_trend_kpis_daily`
 - `gl_content_kpis_daily`
 - `gl_user_influence_7d_overall`
 - `gl_user_influence_7d_by_category`

19.1 How to run both modes safely

Use different run IDs:

- Heuristic run: `run_20260310_heuristic_v1`
- AI run: `run_20260310_ai_v1`

Example:

1. Generate heuristic predictions:
 - `gl_run_agents --run-id run_20260310_heuristic_v1 --analysis-type heuristic`
2. Generate AI predictions:
 - sync: `gl_run_agents --run-id run_20260310_ai_v1 --analysis-type ai`
 - or batch: `gl_batch_prepare/submit/status/download/ingest` with run ID `run_20260310_ai_v1`
3. Run downstream steps separately for each run ID:
 - `gl_compute_horizon`
 - `gl_build_trends`
 - `gl_aggregate_daily`
 - `gl_rank_users`
 - `gl_compute_trend_kpis`
 - `gl_compute_content_kpis`
 - `gl_rank_users_7d`

19.2 Query patterns to compare modes

Compare counts by run and mode:

```
select run_id, analysis_type, count(*) as posts
from gl_post_predictions
group by run_id, analysis_type
order by run_id, analysis_type;
```

Compare trend KPI by day/category:

```
select date_utc, category, analysis_type,
       avg(trend_health_index) as trend_health_avg
from gl_trend_kpis_daily
where run_id in ('run_20260310_heuristic_v1', 'run_20260310_ai_v1')
group by date_utc, category, analysis_type
order by date_utc, category, analysis_type;
```

Interpretation rule:

- Treat `run_id + analysis_type` as a composite experiment key.
- Never blend modes in the same chart unless explicitly comparing them.

20) Worked Example: One Bronze JSON -> Silver Rows -> Gold Rows

This uses a real sample from `rel/data/ig/raw/ig_posts_6mo_5000_mix.jsonl`:

Input Bronze post (abridged):

- `post_id`: IGP_20251001_000001
- `timestamp_utc`: 2025-10-01T13:30:43Z
- `user.user_id`: IGU_00001
- `user.username`: fitcouture6211
- `user.follower_count`: 222
- `caption_text`: lux capsule clean style • UAE
- `hashtags include`: #modestfashion #hijabstyle #uaefashion #abayaedit #
- `media`: reel
- `snapshots include t_day` 0/1/2... with cumulative engagement values

20.1 Silver artifacts created by `sl_load`

Canonical IDs produced:

- `canonical_user_id` = `instagram:IGU_00001`
- `canonical_post_id` = `instagram:IGP_20251001_000001`

Rows written:

1. `sl_users` (1 row)
 - user identity/account metadata, first/last seen timestamps.
2. `sl_user_snapshots` (1 row for publish date)
 - `key`: (`instagram:IGU_00001`, `2025-10-01`)
 - `followers_count`=222, `source`=`bronze_instagram`.
3. `sl_posts` (1 row)
 - post timestamp, content type (`reel` -> `content_type='video'`), caption, ingest metadata.
4. `sl_post_mentions` (0 rows in this sample, since mentions array empty).
5. `sl_tags` (N upserts for unique hashtag norms)
 - e.g. `tag_norm='uaefashion'`, `tag_norm=' '`.
6. `sl_post_tags` (N bridge rows)
 - maps post to each `tag_id` with position.
7. `sl_post_metrics_ts` (one row per snapshot `t_day`)
 - keys like (`instagram:IGP_20251001_000001`, `'age_day'`, `'d:1'`)
 - stores cumulative likes/comments/shares/saves/views.

20.2 Gold prediction artifacts (heuristic run)

Assume:

- `run_id` = `run_20260310_heuristic_v1`

- `analysis_type = heuristic`

`gl_run_agents` writes:

1. `gl_runs` (1 row for run metadata).
2. `gl_post_predictions` (1 row for this post)
 - language likely `ar-en` (Arabic + Latin in caption/hashtags)
 - translated text (heuristic fallback returns source text)
 - fashion flag from token matches (`style`, `abaya`, fashion hashtags)
 - category from heuristic mapping (likely `modestwear/abaya` or fallback `general_fashion/general`)
 - sentiment label and evidence.

20.3 Gold prediction artifacts (AI run)

Assume:

- `run_id = run_20260310_ai_v1`
- `analysis_type = ai`

Batch path writes:

1. `gl_runs` row for AI run.
2. `gl_post_predictions` row for same canonical post, with:
 - model-produced language/sentiment/category/topic,
 - confidence values,
 - evidence JSON from prompt output,
 - taxonomy constrained by canonicalization guards.

20.4 Post-horizon metrics for this post

`gl_compute_horizon --horizon d7` attempts to load day-7 snapshot for the post and compute:

- `followers_at_post` from latest user snapshot on/before publish date.
- `engagement_score = (likes + 2*shares + 1.5*comments) / followers_at_post`.

Row written to:

- `gl_post_metrics_horizon` keyed by (`canonical_post_id`, `run_id`, `horizon_type='d7'`).

20.5 Trend and KPI artifacts this post contributes to

If post is fashion and category is not null:

1. `gl_trend_clusters`
 - cluster for its category (if not already existing for run).
2. `gl_post_cluster_membership`
 - membership row linking post to category cluster.

3. `gl_trend_daily_metrics`
 - contributes to daily `post_count`, `shares_sum`, `engagement_score_*`.
4. `gl_trend_kpis_daily`
 - contributes to breakout/growth/health and concentration/novelty features.
5. `gl_content_kpis_daily`
 - contributes to category-day rates (`*_per_1k`, quality score, viral ratio).
6. `gl_user_influence_daily` and `gl_user_influence_7d_*`
 - contributes to creator influence metrics and ranks.

20.6 Verification SQL for this one post lineage

```
-- Silver lineage
select * from sl_posts
where canonical_post_id = 'instagram:IGP_20251001_000001';

select * from sl_post_tags pt
join sl_tags t on t.tag_id = pt.tag_id
where pt.canonical_post_id = 'instagram:IGP_20251001_000001'
order by pt.position;

select * from sl_post_metrics_ts
where canonical_post_id = 'instagram:IGP_20251001_000001'
order by t_day;

-- Gold lineage per run
select run_id, analysis_type, pred_language, pred_is_fashion, category, subcategory
from gl_post_predictions
where canonical_post_id = 'instagram:IGP_20251001_000001'
order by run_id;

select *
from gl_post_metrics_horizon
where canonical_post_id = 'instagram:IGP_20251001_000001'
order by run_id, horizon_type;
```

This section is the concrete lineage reference for debugging and model-comparison audits.

This document is the canonical implementation narrative for the current branch state.